

AI Assistant Coding

Assignment-1

K. KEERTHANA
2303A52027
BATCH-38

Task 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions)

Use GitHub Copilot to generate a Python program that:

- Prints the Fibonacci sequence up to n terms
- Accepts user input for n
- Implements the logic directly in the main code ➤ Does not use any user-defined functions

Prompt:

write a code to print fibonacci series upto n without using a function

Code:

```
C: > Users > Keerthana1 > OneDrive > Desktop > AILab2.py > ...  
1  # write a code to print fibonacci series upto n without using a function  
2  n = int(input("Enter a number: "))  
3  a = 0  
4  b = 1  
5  for i in range(n):  
6      print(a, end=" ")  
7      c = a + b  
8      a = b  
9      b = c  
10 print()
```

Output:

```
PS C:\Users\Keerthana1\OneDrive\Desktop> & 'c:\Users\Keerthana1\AppData\Local\Programs\Python\Python313\py
nsions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '49245' '--' 'C:\Users\Keerthana1\OneDrive\Desktop\main.py'
Enter a number: 5
0 1 1 2 3
```

Explanation:

This program prints the Fibonacci series.

The user enters a number n. The program starts with 0 and 1. In each loop, it prints the current number and adds the previous two numbers to get the next one. This repeats n times and prints the Fibonacci series.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

- Examine the Copilot-generated code from Task 1 and improve it by:
- Removing redundant variables
- Simplifying loop logic
- Avoiding unnecessary computations ➤ Use Copilot prompts such as: ▪
 - “Optimize this Fibonacci code”
 - “Simplify variable usage”

Prompt:

Optimize the above code by removing unnecessary variables, simplifying the loop, and improving readability.

Code:

```
C:\Users\Keerthana1\OneDrive\Desktop> ASS_AI_1.py > ...
1 # Optimize the above code by removing unnecessary variables, simplifying the loop, and improving readability.
2
3 n = int(input("Enter the number of terms: "))
4
5 if n <= 0:
6     print("Please enter a positive integer")
7 elif n == 1:
8     print(f"Fibonacci series up to {n} term:")
9     print(0)
10 else:
11     print(f"Fibonacci series up to {n} terms:")
12     a, b = 0, 1
13     for _ in range(n):
14         print(a, end=" ")
15         a, b = b, a + b
```

Output:

```
PS C:\Users\Keerthana1\OneDrive\Desktop> c.; cd 'c:\Users\Keerthana1\OneDrive\Desktop'
.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59600' '--' 'C:\Users\Keer
Enter the number of terms: 5
Fibonacci series up to 5 terms:
0 1 1 2 3
```

Explanation:

This program prints the Fibonacci series up to a given number of terms. The user enters a number n. If the number is not valid, the program shows an error message. Otherwise, it starts with 0 and 1 and prints the Fibonacci numbers using a loop. In each step, the next number is found by adding the previous two numbers. The series is printed until n terms are displayed.

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions) Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to generate Fibonacci numbers
- Returns or prints the sequence up to n
- Includes meaningful comments (AI-assisted)

Prompt:

Write a code for printing the Fibonacci series up to n terms using a function and include meaningful comments.

Code:

```
1 # Write a code for printing the Fibonacci series up to n terms using a function and include meaningful comments.
2 def fibonacci_series(n):
3     """
4     This function prints the Fibonacci series up to n terms.
5
6     Parameters:
7     n (int): The number of terms in the Fibonacci series to be printed.
8     """
9     a, b = 0, 1 # Initialize the first two terms of the Fibonacci series
10    count = 0    # Initialize a counter to keep track of the number of terms printed
11
12    # Check if the number of terms is valid
13    if n <= 0:
14        print("Please enter a positive integer.")
15    elif n == 1:
16        print("Fibonacci series up to", n, "term:")
17        print(a) # Print the first term
18    else:
19        print("Fibonacci series up to", n, "terms:")
20        while count < n:
21            print(a, end=' ') # Print the current term
22            a, b = b, a + b    # Update values of a and b to the next two terms
23            count += 1        # Increment the counter
24    # Example usage:
25    fibonacci_series(10) # Print the first 10 terms of the Fibonacci series
```

Output:

```
● PS C:\Users\Keerthana1\OneDrive\Desktop> & 'c:\Users\Keerthana1\AppData\Local\Programs\Python\Python311\Scripts\python.exe' 'c:\Users\Keerthana1\AppData\Local\Programs\Python\Python311\Scripts\debugpy\launcher' '65159' '--' 'C:\Users\Keerthana1\OneDrive\Desktop\task3.py'
Fibonacci series up to 10 terms:
0 1 1 2 3 5 8 13 21 34
○ PS C:\Users\Keerthana1\OneDrive\Desktop> █
```

Explanation:

This program prints the Fibonacci series using a function.

The user enters a number n. The function print_fibonacci checks whether the number is valid. If it is valid, the program starts with 0 and 1 and prints Fibonacci numbers by adding the previous two numbers each time. The series is printed up to n terms

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

Compare the Copilot-generated Fibonacci programs:

- Without functions (Task 1)
- With functions (Task 3)
- Analyze them in terms of:
 - Code clarity
 - Reusability
 - Debugging ease
 - Suitability for larger systems

Prompt:

Non-Modular: Write a code for printing a Fibonacci series up to n terms without using a function.

Modular: Write a code for printing the Fibonacci series up to n terms using a function and include meaningful comments.

Code:

Procedural:

```
G:\Users\Keerthana1\OneDrive\Desktop> task3.py > ...
1 # Optimize the above code by removing unnecessary variables, simplifying the loop, and improving readability.
2 n = int(input("Enter the number of terms: "))
3 if n <= 0:
4     print("Please enter a positive integer")
5 elif n == 1:
6     print(f"Fibonacci series up to {n} term:")
7     print(0)
8 else:
9     print(f"Fibonacci series up to {n} terms:")
10    a, b = 0, 1
11    for _ in range(n):
12        print(a, end=" ")
13        a, b = b, a + b
```

Output:

```
0 1 1 2 3 5 8 13 ...
PS C:\Users\Keerthana1\OneDrive\Desktop> c::; cd 'c:\Users\Keerthana1\OneDrive\Desktop\python.exe' 'c:\Users\Keerthana1\.vscode\extensions\ms-python.debugpy-2025.18.0-w
ana1\OneDrive\Desktop\task3.py'
Enter the number of terms: 8
Fibonacci series up to 8 terms:
0 1 1 2 3 5 8 13
```

Modular:

```
1 # Write a code for printing the Fibonacci series up to n terms using a function
2 # and include meaningful comments.
3
4 def print_fibonacci(n):
5     """
6     Prints the Fibonacci series up to n terms.
7
8     Args:
9         n (int): Number of terms to print in the Fibonacci series
10
11     The Fibonacci series starts with 0 and 1, and each subsequent number
12     is the sum of the previous two numbers.
13     """
14
15     if n <= 0:
16         print("Please enter a positive integer")
17         return
18
19     elif n == 1:
20         print(f"Fibonacci series up to {n} term:")
21         print(0)
22         return
23
24     print(f"Fibonacci series up to {n} terms:")
25
26     # Initialize the first two terms of the series
27     a, b = 0, 1
28
29     # Generate and print each term
30     for _ in range(n):
31         print(a, end=" ")
32         # Update values: a becomes b, b becomes sum of previous a and b
33         a, b = b, a + b
34
35     print() # Print newline at the end
36
37 # Get input from user
38 n = int(input("Enter the number of terms: "))
39 print_fibonacci(n)
```

Output:

Modular:

```
PS C:\Users\keerthana1\OneDrive\Desktop> cd C:\Users\keerthana1\OneDrive\Desktop
python.exe 'c:\Users\Keerthana1\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\python.exe' 'c:\Users\Keerthana1\OneDrive\Desktop\task3.py'
Enter the number of terms: 6
Fibonacci series up to 6 terms:
0 1 1 2 3 5
```

Explanation:

In the **procedural approach**, the Fibonacci series code is written directly in the main program without using any function. The steps run one after another, and this method is simple but not reusable.

In the **modular approach**, the Fibonacci logic is written inside a function. The main program

only calls the function. This makes the code more organized, easy to understand, and reusable.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

Prompt GitHub Copilot to generate:
An iterative Fibonacci implementation
A recursive Fibonacci implementation

Prompt:

Write a code for printing a Fibonacci series up to n terms without using a function.

Write a code for printing the Fibonacci series up to n terms using recursion.

Code:

```
# Write a code for printing a Fibonacci series up to n terms without using a function

n = int(input("Enter number of terms: "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()

# Write a code for printing the Fibonacci series up to n terms using recursion

def fibonacci(n, a=0, b=1, count=0):
    if count < n:
        print(a, end=" ")
        fibonacci(n, b, a + b, count + 1)

n = int(input("Enter number of terms: "))
fibonacci(n)
print()
```

Output:

```
PS C:\Users\Keerthana1\OneDrive\Desktop> c:; cd 'c:\Users\Keert
Programs\Python\Python313\python.exe' 'c:\Users\Keerthana1\.vsco
bs\debugpy\launcher' '53164' '--' 'C:\Users\Keerthana1\OneDrive\
Enter number of terms: 5
0 1 1 2 3
```

Explanation:

This program prints the Fibonacci series in two ways .

In the first part, the Fibonacci series is printed **without using a function**. The user enters the number of terms, and the program uses a loop to print the series by adding the previous two numbers each time.

In the second part, the Fibonacci series is printed **using recursion**. A function is defined that prints one Fibonacci number and then calls itself to print the next one. This process continues until the required number of terms is printed. Both methods produce the same Fibonacci series but use different approaches.